Concurrent FTP Server

Multi-Client File Transfer System

Technology: C Programming (Linux System Programming)

Project Overview

This project is a **Concurrent FTP Server** implemented in **C on Linux** that enables **multiple clients** to connect to a server and **download files concurrently**. The server uses **socket programming** and **process management** to support parallel file transfers.
It provides a strong practical understanding of **Linux networking**, **file subsystem**, **process subsystem**, and building a simple **protocol-driven client-server system**.

Key Features

• Multi-Client Support (Concurrency)

- Handles **multiple clients at the same time**.
- Uses **process-based concurrency** (`fork()`) to create a separate child process per client connection.

• File Transfer over TCP

- Uses **TCP socket** communication to ensure reliable transfer.
- Client requests a file name, server validates and sends it.

• Simple Protocol Design (Header + Data)

- Implements a clean protocol like:
  - Server first sends a header (example: `OK <size>\n` or error message)
  - Then transmits the file in chunks using `read()` and `write()`
- Helps students learn **real protocol design** used in networking systems.

• Linux System Call Usage (Core Focus)

- **File subsystem:** `open, read, close, stat`
- **Network subsystem:** `socket, bind, listen, accept, connect, send, recv`
- **Process subsystem:** `fork, wait, exit`

• Proper Error Handling

- Handles cases like:
  - File not found
  - Client disconnect
  - Invalid requests
  - Permission issues
- Demonstrates real-world concepts:
  - Socket communication
  - Client-server architecture
  - Concurrent programming
  - Linux system calls
  - Transfer reliability

Learning Outcomes

- Deep understanding of **Linux TCP/IP socket programming**
- Practical knowledge of **client-server architecture**
- Hands-on experience with:
  - **fork-based concurrency**
  - **file transfer in chunks**
  - **protocol design (header + payload)**
- Strong command over **system programming in C**
- Confidence to explain networking + OS concepts in interviews

GitHub Repository

🔗Concurrent FTP Server
(Replace your-username with your actual GitHub ID)

Example Usage

Terminal 1 (Server)

```
$ gcc server.c -o server
$ ./server 9090
Server started... Waiting for clients...
Client connected...
Sending file: Demo.txt
Transfer done.
```

Terminal 2 (Client)

```
$ gcc client.c -o client
$ ./client 127.0.0.1 9090 Demo.txt
OK 1234
File received successfully.
```

Multiple Clients (Concurrency Test)

```
$ ./client 127.0.0.1 9090 A.txt
$ ./client 127.0.0.1 9090 B.txt
$ ./client 127.0.0.1 9090 C.txt
```

Present in an Interview

I built a **Concurrent FTP Server in C on Linux** that allows **multiple clients to download files simultaneously** using **TCP socket programming** and **process-based concurrency with `fork()`**.

The server uses Linux system calls such as `socket`, `bind`, `listen`, `accept` for networking and `open`, `read`, `stat`, `close` for file operations.

For concurrency, each client request is handled by a separate child process, ensuring parallel file transfers.

I also designed a simple and clean protocol where the server first sends a header like **`OK <file_size>`** and then transfers the file in chunks.

This project gave me strong hands-on exposure to **Linux networking, file subsystem, process management, and real-world client-server design.**